

Week 3 Technical Report - Data Structures

Project repository (complete source, tests, benchmarks and results):

<https://github.com/RDeibel2025/CTX-Advanced-Algorithms>

Structures: `src/structures/` · demonstration: `examples/week3_demo.py`

Benchmark: `benchmarks/week3_structures_benchmark.py`

Robert Deibel · CSC 5300 Advanced Algorithms · Concordia University Texas · Fall 2026



Complete project repository

<https://github.com/RDeibel2025/CTX-Advanced-Algorithms>

1. Executive Summary

My binary heap and Python's `heapq` run the same algorithm under the same bounds, yet mine was 3.7-7.5× slower at every size from 10^3 to 10^6 : a pure constant factor, Python against C. The rule, where asymptotics tie the constant decides, also held for chaining against probing and for rehashing, though at 10^6 memory adds to the constant. It broke once: the AVL tree's search gap against `dict` tracked $\log_2 n$ almost exactly, an asymptotic difference rather than a constant.

2. Methodology

Environment. Apple M2 Max, 12 cores, 64 GB RAM; macOS 14.5 (arm64); CPython 3.12.4;

`time.perf_counter()`.

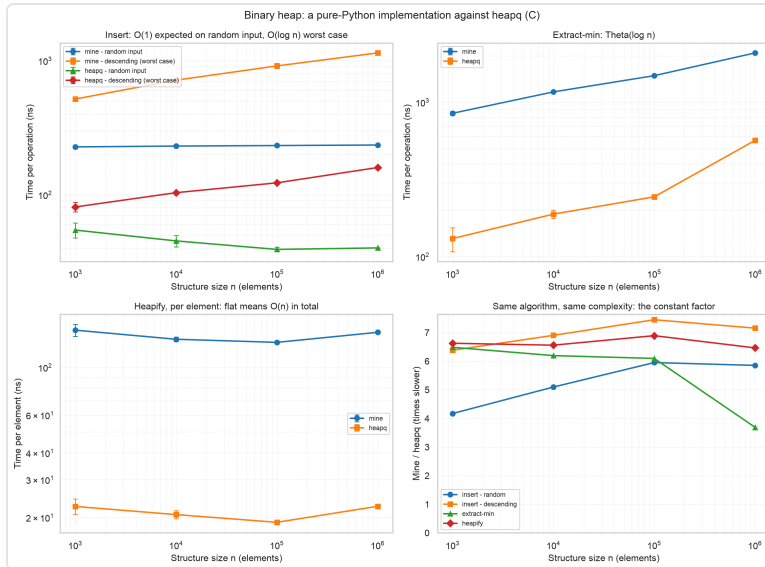
Data and technique. n distinct random integers from $[0, 10^{12})$, seed 2026, $n = 10^3$ to 10^6 . The Week 1 framework gained a `time_operation` method: each run starts from a fresh structure built off the clock, with garbage collection paused. Insert fills an empty structure with all n keys; search and delete visit all n in shuffled order; costs are per operation. The `list` scans a 1,000-to-100-key sample instead, and every structure is verified correct before timing.

Reliability. 5, 5, 3 and 1 measured runs at the four sizes, after 2, 2, 1 and 0 warm-ups. The single run at 10^6 is deliberate, since some cells take seconds, and every CSV row records its count. Median relative SD over the 69 repeated cells: 3.1%.

Supporting studies. Loads 0.1-0.95 in a 65,536-slot table; keys $k \cdot 2^{40}$, sharing one slot at every power-of-two capacity; 131,072 individually timed inserts; AVL heights. `tools/week3_facts.py` prints every figure below from the CSVs.

3. Results

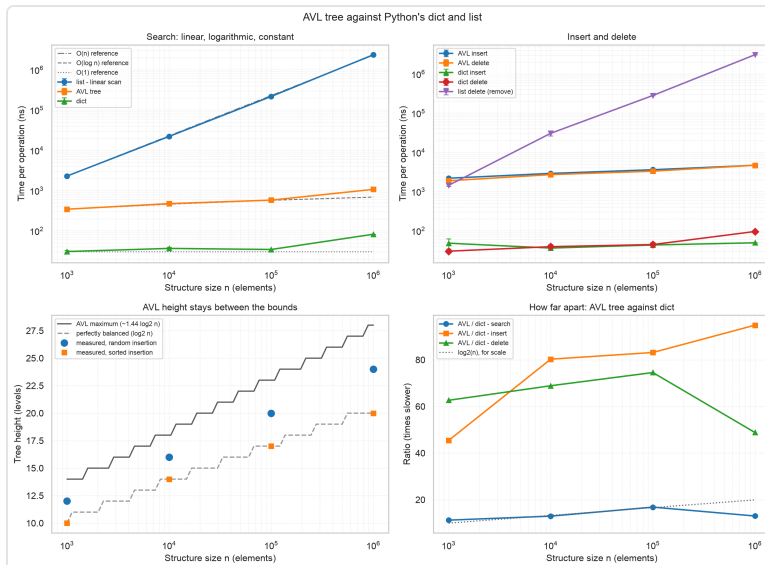
3.1 Heap Performance



Two bounds needed measuring, not asserting. Insert is $O(\log n)$ only in the worst case: on random input mine held flat at 227-235 ns, while descending input, where every key climbs to the root, grew $2.21\times$ across the range ($\log n$ predicts 2.0). Bottom-up heapify held at 131-149 ns per element, so the build is linear. Extraction is the logarithmic operation: 849 to 2,088 ns.

Against heapq the gap was $4.2\text{--}6.0\times$ for insert, $6.4\text{--}7.5\times$ descending, $6.5\text{--}6.9\times$ for heapify and $3.7\text{--}6.5\times$ for extraction, narrowing only at 10^6 . Scaling each 10^5 cost by $\log n$, both extractions came in about 300 ns over (mine 304, heapq 273): one absolute memory penalty, diluting a relative gap.

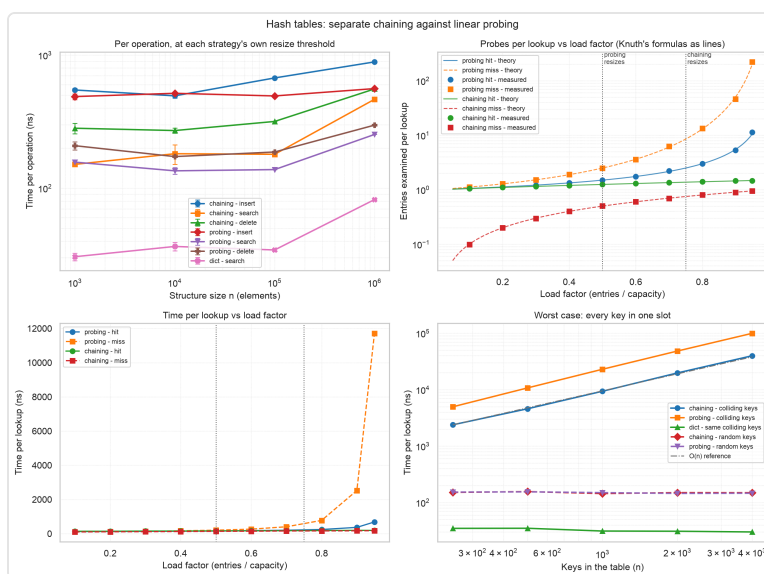
3.2 AVL Tree



Balance held. Random insertion gave heights 12, 16, 20 and 24, about 1.2× perfect (10, 14, 17, 20) and inside the AVL bound (14, 18, 23, 28); sorted insertion, a plain BST's worst case, gave exactly perfect height. Inserts averaged 0.62-0.70 rotations and deletes 0.30 rebalances, but one delete needed 5, a cascade insertion never needs.

Here the data contradicted the spine. I expected the AVL-to-`dict` search gap to exceed what $\log n$ explains. It did not: 11.3×, 13.0× and 16.8× at 10^3 - 10^5 , against $\log_2 n$ of 10.0, 13.3 and 16.6. The tree spends 36-38 ns per node visited (9.19 to 15.95 per search); a whole `dict` lookup costs 30.5-36.5 ns. The search gap is the depth, the asymptotic difference itself. (At 10^6 it falls to 13.0× as `dict` slows to 82 ns.) The constant shows in insert and delete: 45-95× slower than `dict`, 4.5-6.3× per level, the price of recursion, height updates and rotations.

3.3 Hash Table



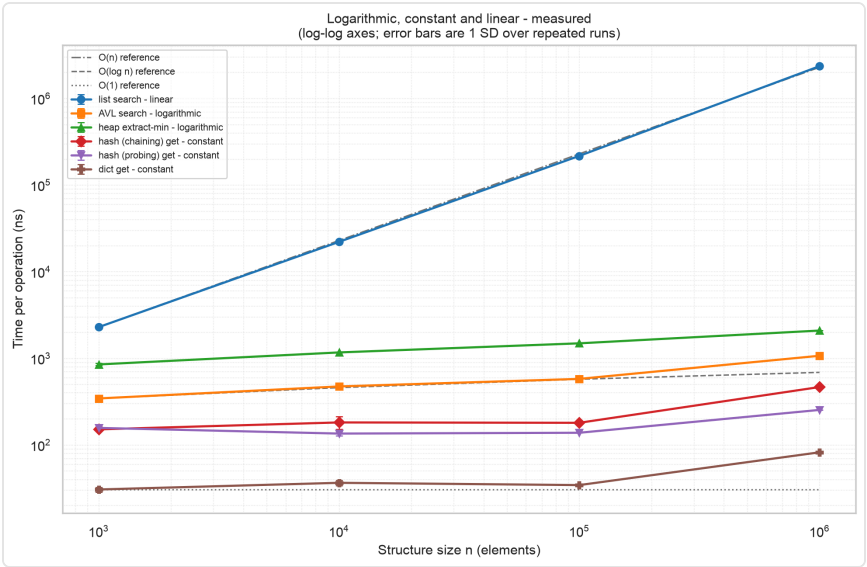
Measured probe counts sat on Knuth's formulas: median error 0.3%, worst 10.7% at $\alpha = 0.95$. Time followed. Through $\alpha = 0.3$ the strategies stayed within 21% of each other, then diverged: by 0.95 a chaining miss had risen 1.7× (99 to 174 ns), a probing miss 110× (107 ns to 11.7 μ s), because a probing miss crosses the whole cluster.

Hence two thresholds: chaining resizes at 0.75, where a miss costs under one comparison, and probing at 0.5, where it costs 2.5 probes rather than 8.5. At those thresholds probing won the main matrix, searching faster at three sizes of four (up to 1.84×) and deleting faster at all four, paid for in memory: at 10^4 it held twice chaining's slots.

The engineered worst case made $O(1)$ a measured $O(n)$: with every key in one slot, lookup time doubled with n (ratios 1.93-2.16), reaching 268× (chaining) and 683× (probing) the random-key cost at $n = 4,000$. `dict` held 30-35 ns on the same keys: CPython mixes the hash's high bits into its probe sequence, and my power-of-two mask discards them.

Growth at 10^6 is not algorithmic. Rebuilt from the same seed, the 10^6 chaining table sits at nearly the 10^3 table's load (0.477 against 0.488) and examines as many entries per hit (1.24 against 1.26), yet takes 465 ns against 151: same work, triple the time, consistent with outgrowing the processor's caches.

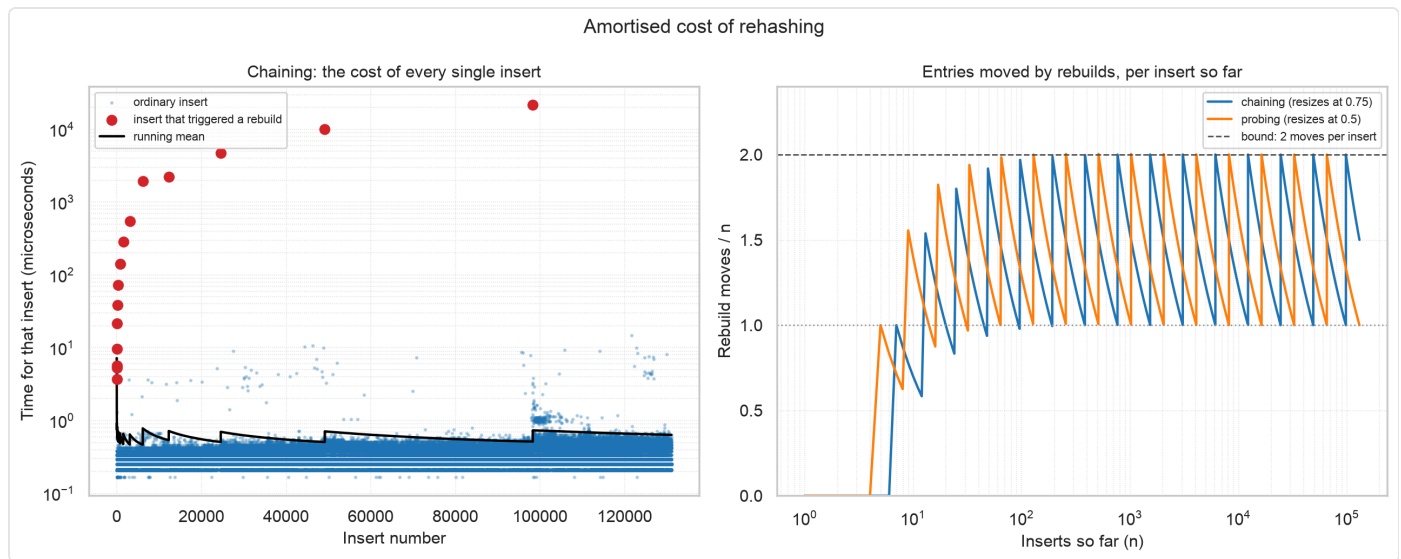
3.4 Comparison Table



Structure, operation	Big-O	ns, 10 ³	ns, 10 ⁶	Measured
Heap insert, random	O(1) expected, O(log n) worst	227.4	234.9	O(1)
Heap extract_min	O(log n)	848.5	2,088.0	O(log n)
Heap heapify, per element	O(n) total	149.1	145.7	O(1)
heapq extract_min	O(log n)	130.8	565.7	O(log n)
AVL insert	O(log n)	2,180.4	4,663.1	O(log n)
AVL search	O(log n)	343.5	1,068.3	O(log n)
AVL delete	O(log n)	1,879.9	4,662.0	O(log n)
dict get	O(1) average	30.5	82.0	O(log n)*
list search	O(n)	2,288.2	2,355,602.1	O(n)
Chaining get	O(1) average	151.0	465.2	O(log n)*
Probing get	O(1) average	156.9	253.2	O(log n)*

16 of the 23 series in `comparison_table.csv` (every size, run count, slope and growth ratio) match their expected class. All seven mismatches (*) are O(1) hash-table series growing at 10⁶, which §3.3 traces to memory, not work; the classifier's verdict stands unedited.

4. Amortized Analysis



Rehashing is the spine's payoff: a constant not paid at once but smeared across operations. Growing a chaining table to 131,072 keys took 15 rebuilds. The largest took 21.4 ms, the time of 73,672 median inserts, and rebuilds were half of all insert time, yet the mean held at 631 ns per insert.

The potential argument for doubling tables (CLRS §16.4) bounds entries moved at under 2 per insert. The measured ratio saws between 1 and 2 as rebuilds land and never exceeded 2; it ended at 1.50 for chaining and 1.00 for probing, which stopped at a sawtooth low.

Amortized $O(1)$ is a promise about the total. A throughput-bound system collects on it; a latency-bound one still meets the 21 ms insert.

5. Practical Recommendations

- **Prefer the built-ins.** `heapq` and `dict` beat equal-complexity Python by 3.7-7.5×; hand-write a structure only for an operation they lack.
- **Heaps** for repeated min or max (schedulers, Dijkstra, top-k), built with `heapify`.
- **Balanced trees** when order matters: range queries, sorted iteration, nearest key, a guaranteed worst case. Point lookups cost them about $\log_2 n$ node visits.
- **Hash tables** for point lookups, with probing resized near 0.5, chaining near 0.75, and the hash mixed before masking: my low-bit mask let colliding keys cause a 683× slowdown that `dict` shrugged off. Where memory is tight, chaining tolerates high load.
- **Hybrids.** Java 8's `HashMap` turns long chains into balanced trees, capping the worst case at $O(\log n)$; an LRU cache pairs a hash table with a linked list; a heap indexed by a hash table gains $O(\log n)$ decrease-key.

6. Conclusion

Big-O predicted the shape of every curve and the size of almost no gap. It separated the list from the rest, 2,205× slower than the AVL tree at 10^6 ; within a class, the language set the heap's gap, the resize threshold set probing's lead, and memory set the 10^6 step. Scalable algorithms inherit these costs: Dijkstra runs at its priority queue's speed, a hash join at its table's. Where the asymptotic term did decide, tree against `dict`, the tree buys what hashing cannot: order. Every choice is a trade: order for a factor of $\log n$, occasional 21 ms pauses for constant average inserts, spare memory for shorter probes.

References

- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4th ed., Ch. 6, 11, 12-13 (Problem 13-3, AVL trees), §16.4
- Amakobe. *Advanced Computational Algorithms*, 2nd ed., Ch. 3
- Knuth. *The Art of Computer Programming*, Vol. 3, 2nd ed., §6.4

AI use: Claude (Claude Code) drafted the code, tests and this report from my specification, and every measurement comes from real runs on my machine ([full disclosure](#)).